Project Sprint #3

The Solitaire game is described in CS449HomeworkOverview.docx. You should read the description very carefully.

Your submission must include the GitHub link to your project and you must ensure that the instructor has the proper access to your project. You will receive no points otherwise.

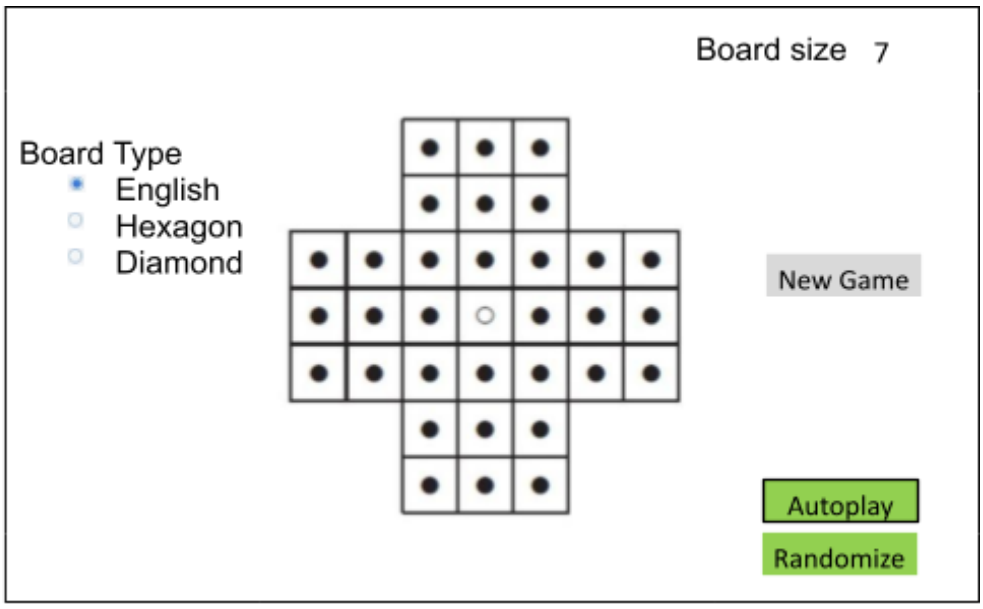
GitHub link: https://github.com/jecalvil/Peg-Solitaire-Project

This sprint assumes that all the features that support a human player to play a manual or automated Solitaire game is already implemented. The minimum features include **choosing the game mode (manual or automated), choosing the board size, setting up a new game, making a move (in a manual or automated game), randomize the state of the board** and **determining if a manual or automated game is over**.

The main task of this assignment is adding the feature of recording a game into a text file (or a lightweight database) and replaying from text file. The following is the complete GUI layout.

VIDEO LINK:
https://umsystem.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=46181288-f81f-4314-ba7a-b4ff0174b1ca&start=0

Grader Note: In the video I said "English Standard" by accident whenever I clicked Hexagon. This was just a mistake, I meant to say Hexagon. I also forgot to mention the key point, which was that the only fundamental difference between manual and automatic is the move generation (manual is mouse click and automatic is random from a pool of legal moves). I also wanted to mention that the automatic/manual mode is a toggle so you can go back and forth between the modes in the same game. Also whenever you load the game, you can continue to play right where it leaves off (assuming the game isn't over). Also, the randomization doesn't always end the game, we just got unlucky and randomized the board into a state that ended the game. Thank you, I tried my best recording the video, but there are only so many details I can mention (especially without messing up).



Deliverables: expand and improve your submission for sprint 2.

1. **Demonstration (9 points)**
   Submit a video of no more than five minutes, clearly demonstrating the following features.
   (a) A manual game until the game is over for English board
   (b) A manual game until the game is over for Hexagon board
   (c) An automated game until the game is over
   (d) A manual game with periodic randomization of states until the game is over
   (e) Some automated unit tests for the manual game mode
   (f) Some automated unit tests for the automated game mode

   In the video, you must explain what is being demonstrated.

2. **Summary of Source Code (1 points)**

Source code file name	Production code or test code?	# lines of code
board.py	Production	320
components.py	Production	135
game_logger.py	Production	69
peg_solitaire_gui.py	Production	772
sound_manager.py	Production	34
test_peg_solitiare.py	Test	579
PressStart2P-Regular.ttf, images/, music/, /saved_games	Other (font, images/icons, music)	N/A
	Total	1909

Images: green_background.jpg, muted.png, red_background.png, volume.png, wood_background.png
Music: mondamusic-retro-arcade-game-music512837.mp3
/saved_games: Just includes some JSON save files.

You must submit all source code via github to get any credit for this assignment.

3. **Production Code vs User stories/Acceptance Criteria (3 points)**

Summarize how each of the user story/acceptance criteria is implemented in your production code (class name and method name etc.)

User Story ID	User Story Name
1	Choose a board size and type

2	Choose the game mode of a chosen board (manual/automated)
3	Start a new game of the chosen board size and game mode
4	Make a move in a manual game
5	A manual game is over
6	Make a move in an automated game
7	An automated game is over
8	Randomize the state of the board during a manual game

Acceptance Criteria/Stories

- Story 1: Choose a board size and Type
    - AC 1.1: Successful pick of valid board size
        - Given: setup screen is display edwith default size of 7
        - When: I select the 5, 7, or 9 radio button
        - Then that radio button is shown as selected
        - And: the game is created with that size
    - AC 1.2: Only odd board sizes are offered
        - Given: setup screen is displayed
        - When: Size options are viewed
        - Then: Only 5, 7, and 9 are offered as radio button options
        - And: Even sizes such as 6 or 8 are not shown
    - AC 1.3 Invalid sizes are rejected
        - Given: set screen is displayed
        - When: I click to change the board size
        - Then: Only UI allows radio button options, so out of range values, decimals, symbols cannot be chosen
        - And: if an invalid size is passed in the game logic regardless, then it still defaults back to 7
    - AC 1.4: Default board type (2.1 in other doc)
        - Given: the game has been launched
        - When: The setu p screen is displayed
        - Then: English, Hexagon, and Diamond are offered
        - And: English is chosen by default
    - AC 1.5 (Selecting different board type (2.2 in other doc)
        - Given: setup screen is displayed with English selected
        - When: Another board type is chosen, such as Hexagon (or Diamond)
        - Then: Hexagon becomes chosen with english no longer selected
        - And: The next game will be created as a hexagon board
    - AC 1.6: Only one chosen board type at a type
        - Given: setup screen is displayed and diamond is selected
        - When: I select Hexagon (or any other type)
        - Then: Only that type is selected
        - And: The board size remains the same
- Story 2: Choose the game mode of a chosen board
    - AC 2.1: Default game mode is manual
        - Given: I create a new game
        - When: the game screen is displayed
        - Then: The autoplay buttons reads “Autoplay: Off”

        - And no moves are made until I make one
    - AC 2.2: Switching to automated mode
        - Given a manual game is in progress and there exists at least one legal move
        - When I click the aotplay button
        - Then the button will read Autoplay ON
        - And the computer will make moves automatically
    - AC 2.3: Switching back to manual mode
        - Given: Autoplay is running
        - When: I click the autoplay button again (toggle)
        - Then the button says AutoPlay OFF and the automatic moves stop
        - And I can make moves manually (of course assuming there are moves left)
    - AC 2.4 Automated mode ends with the game
        - Given: Autoplay is going
        - When the game ends (win or lose)
        - Then: Autoplay switches itself off/hides
- Story 3 Start a new game of the chosen board size and game mode
    - AC 3.1: Starting a new english game
        - The board is of size 8 and the type is english
        - When a game is created
        - Then a cross shaped board with 33 positions is shown
        - And 32 contain a peg and only the center is empty
    - AC 3.2: Starting a game of another type/size
        - Given: The type is Diamond as the size is 9 for example
        - When a game is created
        - Then a diamond shaped board with 41 positions is shown
        - And 50 contain a peg and the center is empty
    - AC 3.3: Restarting a game (replaces old 5.5 for restart)
        - Given a game is in progress or over
        - When I click Restart
        - Then a new board is created with same type and size
        - And any game over message and peg selection are cleared
- Story 4: Make a manual move in a game
    - AC 4.1 Selecting a peg that has a legal move
        - Given: manual game is in progress
        - When I click a peg with at least a legal jump
        - Then the peg is shown as selected
        - And each empty hole that can legally be jumped is highlighted
    - AC 4.2: Making a valid jump
        - Given a peg is selected and highlighted legal move hole is two pos away (orthog or diagonal) with peg between then
        - When the hole is clicked
        - Then the peg moves into the hole and the jumped peg is removed
        - And the starting hole becomes empty
    - AC 4.3 Rejecting illegal destinations
        - Given a peg is selected
        - When I click a hole or position that is not a legal jump
        - Then no move is made and the board is unchanged

        - And a message is shown saying its not allowed
    - AC 4.4 Rejecting an empty hole or peg with no legal moves
        - Given a manual game is in progress and nothing is selected
        - WHne I click an empty hole or peg with no legal jumps
        - Then nothing is selected and the board is unchanged
        - And a message says no move is available from that positions (or something similar)
    - Ac 4.5 Deselecting a peg
        - Given a peg is selected
        - When I lick the peg again
        - Then the peg is no longer selected and highlight disappears
        - And the board is unchanged
    - AC 4.6 Selecting another legal peg
        - Selecting another legal peg
            - Given a peg is selected
            - When I click another peg with a legal move
            - Then a new peg is selected and legal move highlighted
            - And previous peg and highlight is no longer shown
- Story 5: A manual game is over
    - AC 5.1:
        - Given manual game is in progress and more than one peg remains'When I mae a move that leaves no legal jump
        - Then the game is over and a message is shown and pegs remaining are displayed'
        - And rating is shown (very good with 2, good with 3, average with 4 or more)
        - And I can start a new game with Restart or Back
    - AC 5.2: Winning the game
        - Given a manual game is in progress
        - WHen I make a move that leaves exactly one peg
        - Then a win message is shown with a final peg count of 1
        - And my rating is Outstanding and I can start a new game
    - AC 5.3 Game continues with legal moves exit
        - Given a manual game is in progress
        - When i make a move and at least one peg still has a legal jump
        - Then the game does not end and no game over message is shown
    - AC 5.4 Moves are disabled after game ends
        - Given game over or win message is shown
        - When i click a peg or empty hole
        - Then nothing is selected and no moves made
        - And the board stas as it was when the game ended
    - AC 5.5:
        - Starting a new game after game over
        - Given the game over message is displayed
        - When I click RESTART or BACK
        - Then a new board is made with selected size and type
        - And the game over message is cleared and i can play
- Story 6: Make a move in an automated game
    - AC 6.1 Computer makes moves on its own (was 9.1 in other doc)
        - Given: Autoplay is on and at least one legal move exists

        - When the next move is due
        - Then the computer highlights the peg and makes the jump
        - And each move is visible on the board
    - AC 6.2 Automated moves are legal and random
        - Given: autoplay is running
        - When the computer chooses its next move
        - Then the move is legal (orthog or diag) over a peg into an empty hole two spaces away
        - And it is picked at random from all available legal movies
    - AC 6.3 Manual input is blocked during autoplay (was 9.4)
        - Given autoplay is running
        - When I click a peg or an empty hole on the board
        - Then no peg is selected and no move is amde
        - And autplay continues unaffected
    - 6.4 Autoplay works on every board type and size (was 9.5)
        - Given new game on any tupe (English, Hexagon, Diamond) and size (5, 7, 9)
        - When I turn autoplay on
        - Then; autoplay plays until the game ends using the same rules as manual play
    - 6.5 Automated moves are recorded
        - Given autoplay marks a mode
        - When the move is made
        - Then it is added to the games move history the same way a manual move is
- Story 7: An automated game is over
    - AC 7.1 Autoplay ends when no legal moves remain (was 9.3)
        - Given autoplay is running and no more than one peg remains
        - When a move leaves no legal jump
        - Then autoplay stop and a game over message is shown
        - And the epgs remaining and my rating are displayed
    - AC 7.2 Starting a new game after autoplay
        - Given an automated game has ended
        - When I click restart or back (back and go back to a fresh game)
        - Then a new board is created and the game over message is cleared
        - And: I can play manually or turn autoplay on again
- Story 8: Randomize the state of the board during manual game
    - AC 8.1 Shuffling the pegs:
        - Given game is in progress
        - When I click shuffle pegs
        - Then the pegs and empty spaces are rearranged across the playable spaces
        - And the number of pegs and empty spaces is unchanged and the invalid corners stay invalid
        - And any peg selection is cleared
    - AC 8.2 Random layout
        - Given a game is in progress
        - When I click random layout
        - Then every playable piece then becomes a peg or empty space at random, with at least one empty space (to ensure its playable)
        - And the board shape/size is unchanged
    - AC 8.3: randomization/shuffle is recorded

        - Given I shuffle or randomize the board
        - When the board changes
        - Then the new layout is stored in the game history so a saved game can be replayed
    - AC 8.4 Randomization is unavailable after game over
        - Given the game is over (win or lose)
        - When I look at the control buttons
        - Then the shuffle and random layout buttons are not shown and cannot be clicked
- Story 9: Save and replay a recorded game
    - AC 9.1: Saving a game
        - Given a game is in progress
        - When I click save game
        - Then a json file wt the board type, size and move history is written to the saved_games folder
        - And a confirmation message is shown
    - AC 9.2 Loading a saved game
        - Given I am on the game screen
        - When I click load game and picka saved file
        - Then the board type and size from the fle are restored
        - And the board resets adn the saved actions are replayed one at a time while a replay message is shown
        - And a message shows when the replay finished
    - AC 9.3 Replay includes randomization
        - Given a saved game contains shuffle or random layout events
        - When I lad it
        - Then the board shows each layout change at the same point in the replay with the later moves applied after
    - AC 9.4 Controls are disabled during a replay
        - Given a saved game is being replaced
        - When I look at the buttons and click them
        - Then the save, load, autoplay, shuffle, random layout, and hint buttons are hidden and ignore clicks
        - And board clicks do nothing until the replay finishes

User Story ID	AC ID	Class Name(s)	Method Name(s)	Status (complete or not)	Notes (optional)
1	1.1	Board, PegSolitaireApp, RadioButton	Board.__init___, Board.is__valid_size; PegSolitaireApp.options_screen, RadioButton.draw, RadioButton.check_click	Complete	Pick a valid size. Radio buttons for 5,7,9, default as 7
	1.2	Board, PegSolitaireApp, RadioButton	Board.__init__, Board.is_valid_size; PegSolitaireApp.options_screen;RadioButton.draw, RadioButton.check_click	Complete	Only odd sizes offered, no even size options exist in UI

	1.3	Board, PegSolitaireApp, RadioButton	Board.__init, Board.is_valid_size; PegSolitaireApp.options _screen;	Complete	Invalid input rejected, UI blocks bad values. Board falls back to default 7 if handed one
	1.4	PEgSolitaireApp, RadioButton	PEgSolitaireApp.options _screen; RadioButton.draw, RadioButton.check_clic k	Complete	Default type is English (was 2.1)
	1.5	PegSolitaireApp, RadioButton	PegSolitaireApp.options _screen; RadioButton.draw, RadioButton.check_clic k	Complete	Selecting another type (was 2.2)
	1.6	PegSolitaireApp, RadioBUtton	PegSolitaireApp.options _screen	Complete	Only one type selected at a time (was 2.3).
2	2.1	Peg Solitaire App, MenuButton	PegSolitaireApp.game_s creen (autoplay_enabled, autoplay_button)	Complete	Default mode is manual (Autoplay off). Mode is switched with the autplay button on the game screen, not on the setup scren
	2.2	PegSolitaireApp, Menu Button	PegSolitaireApp.game_s creen;MenuButton.check _click, MenuButton.update_text	Complete	Clicking the button switches to automatic mode, label becomes autoplay: on
	2.3	PegSOlitaireApp, MenuButtn	PegSolitaireApp.game_s creen; MenuButton.check_click , MenuButton.update_text	Complete	Clicking again returns to manual and automoves stop
	2.4	PegSolitaireApp	Board.__init__, Board.is_english_corner; PegSolitaireApp.options	Complete	Autoplay switches itself off when the game ends (win/loss)
3	3.1	Board, pegSolitaireApp	Board.__init__, Board.is_english_corner; PegSolitaireApp.options _screen, pegSolitaireApp.game_s creen	Complete	New English 7 game: 33 spaces, 32 pegs, center empty hole
	3.2	Board, PegSolitaireApp	Board.__init__, Board.is_diamond_corne r, Board.is_hexagon_corne r; PegSolitaireApp.game_s creen	Complete	Other types ansd sizes
	3.3	Board, PegSolitaireApp, MenuButton	PegSolitaireApp.game_s creen (RESTART and BACK handling);	Complete	Restarting builds fresh board with same type and size,

			MenuButton.check_click; Board.__init		clears self.aciton_history
4	4.1	Board, PegSolitaireApp	Board.get_legal_moves, Board.is_valid_move; PegSolitaireApp.game_screen	Complete	Selected peg turns yellow, legal moves highlighted with greenr rings, move appended to action history
	4.2	Board, PegSolitaireApp	Board.make_move, Board.is_valid_move; PegSolitaireApp.game_screen	Complete	Orthog/Diag jmps, jumped peg removed, move appended to action history
	4.3	Board, PegSolitaireApp	Board.make_move, Board.is_valid_move; PegSolitaireApp.game_screen	Complete	Board unchanged and move not allowed shown, in current mode, peg stays selected, might change to become unselected or might keep it (small preference detail)
	4.4	Board, PegSolitaireApp	Board.get_legal_moves; PegSolitaireApp.game_screen	Complete	No move available fromt that position message, nothing selected
	4.5	PegSolitaireApp	PegSolitaireApp.game_screen (selection/deselection)	Complete	Clicking selected peg deselects it
	4.6	PegSolitaireApp	PegSolitaireApp.game_screen (selection/deselection)	Complete	Clicking another moveable peg moves the selection and highlights to it
5	5.1	Board, PegSolitaireApp, MenuButton	Board.game_over_check, Board.count_pegs, Board.get_rating; PegSolitaireApp.game_screen; MenuButton.check_click	Complete	Game over title with rating/pegs left, new game via restart or back
	5.2	Board, PegSolitiareApp	Board.check_win, Board.get_rating; PegSolitaireApp.game_screen	Complete	You won title and outstanding rating
	5.3	Board	Board.game_over_check	Complete	Game continues while a legal jump exists
	5.4	PegSolitaireApp	PegSolitaireApp.game_screen	Complete	Grid clicks are ignored once game over text is set
	5.5	PegSolitaireApp, MenutButton	PegSolitaireApp.game_screen; MenuButton.check_click	Complete	New game via Restart (same settings) or back (new settings)

6	6.1	Board, PegSolitaireApp	select_random.move, Board.make_automated_move, Board.make)move; PegSolitaireApp.game_screen	Complete	Every 500 ms: first highlights chosen peg (pending_auto_move)
	6.2	Board	Board.select_random_move, Board.get_legal_moves, Board.is_valid_move, Board.make_automated_move	Complete	Radnom pick from all legal moves (9.2 prev)
	6.3	PegSolitaireApp	PegSolitaireApp.game_screen (grid click handling, autoplay_enabled)	Complete	Prev 9.4, grid clicks ignored while autoplay is on, autoplay keeps running
	6.4	Board, PegSolitaireApp	Board.select_random_move, Board.make_automated_move, Board.game_over_check; PegSolitaireApp.game_screen	Complete	Same logic on every board type and size (prev 9.5)
	6.5	PegSolitaireApp, GameLogger	PegSolitaireApp.game_screen (action_history.append); GameLogger.save_game	Complete	Automated moves are recorded like manual ones (earlier bug fix)
7	7.1	Board, pegSolitaireApp	Board.game_over_check, Board.get_rating; PegSolitaireApp.game_screen	Complete	No legal moves left, autoplay turns off, game over and rating withs pegs left are show, prev 9.3
	7.2	Board, pegSolitaireApp	Board.check_win, Board.get_rating; PegSolitaireApp.game_screen	Complete	One peg elft, saying yuo won and outstanding rating, same checks as manual play
	7.3	PegSolitaireApp, MenuButton	PegSolitaireApp.game_screen; MenuButton.check_click	Complete	New game after an automated game via RESTART or back; game over message cleared (prev 9.6)
8	8.1	Board, pegSolitaireApp	Board.shuffle_board, Board.get_non_invalid_moves, Board.count_pegs; PegSolitaireApp.game_screen	Complete	Shuffle pegs keeps the same number of pegs and empty spaces and moves them around, invalid corners untouched.
	8.2	Board, PegSolitaireApp	Board.randomize_layout, Board.get_non_invalid_moves;	Complete	Random Layout refills every plaable space randomly with a peg or emty (at

			PegSolitaireApp.game_screen		least one empty, to maintain playability as although the odds are low, a board with full pegs would just end the game).
	8.3	PegSolitaireApp, GameLogger	PegSolitaireApp.game_screen; GameLogger.save_game	Complete	Each shuffle/randomize is stored as a grid snapshot in action history
	8.4	PegSolitaireApp	PegSolitaireApp.game_screen	Complete	Shuffle and random layout buttons are hidden and unclickable after game over.
	8.5	Board, PegSolitaireApp	Board.game_over_check , Board.check_win; PegSolitaireApp.game_screen	Complete	End of game check runs every frame so it applies together randomize
	9.1	GameLogger, PegSolitaireApp	GameLogger.save_game ;PegSolitaireApp.game_screen (SAVE GAME)	Complete	Writes JSON (boad type, size, action history) into save_games/.
	9.2	GameLogger, PegSolitaireApp	GameLogger.load_game ; PegSolitaireApp.game_screen (LOAD GAME, tkinter file dialog)	Complete	Reads JSON and turns coordinate list back into tuples, saved actions are queued in replay_queue
	9.3	Board, PegSolitaireApp	Board.make_move; PegSolitaireApp.game_screen (replay_queue, REPLAY_DELAY)	Complete	Replays one saved action every 500 ms (moves and shuffle/randomize snapshots) so the game plays back step by step, messages saying replaying saved game shows while it runs and replay finished at the end
	9.4	PegSolitaireApp	PegSolitaireApp.game_screen (replay_queue checks on drawing and click handling)	Complete	While a replay is running, the sabe, load, autoplay, shuffle, random layout and hint buttons are hidden and ignore clicks, and board clicks are paused too

4. Tests vs User stories/Acceptance Criteria (2 points)

Summarize how each of the user story/acceptance criteria is tested by your test code (class name and method name) or manually performed tests.

User Story ID	User Story Name
1	Choose a board size and type
2	Choose the game mode of a chosen board (manual/automated)
3	Start a new game of the chosen board size and game mode
4	Make a move in a manual game
5	A manual game is over
6	Make a move in an automated game
7	An automated game is over
8	Randomize the state of the board during a manual game

4.1 Automated tests directly corresponding to some acceptance criteria

User Story ID	Acceptance Criterion ID	Class Name (s) of the Test Code	Method Name(s) of the Test Code	Status (Pass/Fail/In Progress)
1	1.1	TestBoardSize	test_valid_board_size, test_valid_sizes_accepted_as_text	Pass
	1.2	TestBoardSize	test_invalid_board_size, test_other_even_sizes_rejected	Pass
	1.3	TestBoardSize	test_out_of_range_rejected, test_non_numeric_rejected	Pass
	1.4-1.6	TestBoardSize (logic only)	test_unknown_board_type_rejected; selection itself is manual	Pass (logic); GUI manual
3	3.1	TestNewGame	test_english_7_counts, test_english_other_sizes	Pass
	3.2	TestNewGame	test_diamond_counts, test_hexagon_counts, test_center_only_empty, test_invalid_size_falls_back_to_default	Pass
4	4.1	TestMakeMove	test_ac_4_1_peg_has_legal_moves	Pass
	4.2	TestMakeMove	test_ac_4_2_valid_jump_updates_board	Pass
	4.3	TestMakeMove	test_ac_4_3_illegal_move_leaves_board_unchanged	Pass
	4.4	TestMakeMove	test_ac_4_4_empty_or_immobile_position_has_no_legal_moves	Pass
	4.5	TestMakeMove	test_ac_4_5_deselect_selected_peg (skipped placeholder)	Pass
	4.6	TestMakeMove	test_ac_4_6_select_another_legal_peg (skipped placeholder)	Pass
5	5.1	TestGameOver	test_ac_5_2_one_peg_remaining_is_a_win	Pass
	5.2	TestGameOver	test_ac_5_2_one_peg_remaining_is_a_win	Pass
	5.3	TestGameOver	test_ac_5_3_game_continues_when_legal_moves_remain	Pass

	5.4	TestGameOver	test_ac_5_4_moves_disabled_after_game_over (skipped placeholder)	Pass
	5.5	TestGameOver	test_ac_5_5_restart_after_game_over (skipped placeholder)	Pass
6	6.1	TestAutomatedMoves	test_automated_move_updates_board	Pass
	6.2	TestAutomatedMoves	test_selected_random_move_is_valid	Pass
	6.4	TestAutomatedMoves	test_automated_game_loop_until_over (Diamond 5), test_automated_move_updates_board (Hexagon 9)	Pass
7	7.1	TestAutomatedMoves	test_automated_game_loop_until_over	Pass
	7.2	TestGameOver	test_ac_5_2_one_peg_remaining_is_a_win (shared win check)	Pass
8	8.1	TestRandomizedBoard	test_shuffle_board_preserves_counts_and_corners	Pass
	8.2	TestRandomizedBoard	test_randomize_layout_generate_playable_board	Pass
9	9.1	TestGamePersistence	test_save_game_creates_valid_json	Pass
	9.2	TestGamePersistence	test_load_game_restores_action_history_and_tuples	Pass
	9.3	TestGamePersistence	test_replay_with_shuffle_and_randomize_snapshots (data only; step-by-step playback is manual)	Pass

4.2 Manual tests directly corresponding to some acceptance criteria (if any)

User Story ID	**Acceptance Criterion ID**	**Test Case Input**	**Test Oracle (Expected Output)**	**Notes**
1	1.1-1.3	Open setup screen and click each size radio button	Only 5, 7, and 9 shown, 7 selected by default, chosen size is used for game	Pass (carried from sprint 2)
	1.4-1.6	Open setup screen, click hexagon then diamond	English selected by default, clicked type becomes the only one selected, size unchanged	Pass (carried from Sprint 2)
2	2.1	Start any game and look at the autoplay button	Shows Autoplay off and no automatic moves happen	Pass
	2.2	Click autoplay	Label changes to autoplay on and computer starts making moves	Pass
	2.3	Click autoplay again while it is running	Label returns to off and auto moves stop, can then do manual moves	Pass
	2.4	Let autoplay run until game ends	Autoplay label returns to OFF and the game over/win title shows	Pass
3	3.3	Make a few moves and click restart	Fresh board of the same time and size, messages cleared	Pass

4	4.1	Click peg that has legal jump	Peg turns yellow and legal holes show green ring	Pass
	4.2	Click highlighted hole	Peg jumps, jumped peg disappears, start hole empty	Pass
	4.3	Select a peg, then click a hole that is not a legal jump	Board unchanged, move not allowed shown	Pass
	4.4	Click empty peg hole or peg with no legal jumps	Nothing selected, no available move message	Pass
	4.5	Select a peg then click it again	Peg highlights, but then unhighlights upon reclick	Pass
	4.6	Select a peg, then click a different movable peg	New peg is selected old highlight is removed	Pass
5	5.1	Play manu game until no moves remain with more than one peg	Game over, rating with pegs left, restart/back work	Pass
	5.2	Play a manual game down to one peg	You Won message and outstanding rating	Pass
	5.4	After the game ends, click pegs and holes	Nothing is selected, board unchanged	Pass
	5.5	After the game ends click restart or back thenre start	New board, game over message cleared	Pass
6	6.1	Turn autoplay on	Peg is highlighted, then its jumps repeats about every half second or second	Pass
	6.3	While autoplay runs click a peg and a hole	No selection and no moves, autoplay continues	Pass
7	7.1	Let autoplay run until its stop	Game over rating with pegs left, autoplay off	Pass
	7.3	After an autoplay game ends, click restart	New board message cleared, manual and autoplay both usable	Pass
8	8.1	Click shuffle pegs mid game	Same number of pegs, positions change, corners stay valid	Pass
	8.2	Click random layout mid game	New random layout on same board and shape	Pass
	8.3	Shuffle make a move, save game, then load game	Loaded board matches saved game	Pass

	8.4	Reach game over and look at the right hand buttons	Autoplay, shuffle, random layout and hint buttons should be hidden	Pass
9	9.1	Click save game during a game	Message should say that the game was saved to /saved_games, a JSON file with the corresponding moves played appears in that folder	Pass
	9.2	Click load game	Board resets, then the saved action play back one at a time, replaying the saved game message shows and replay finished when it finishes	Pass
	9.3	Save a game that includes a shuffle or random layout, then load it	Layout change appears in the same place during the replay and later moves follow	Pass
	9.4	Start a replay and try to click save, load, autoplay, shuffle, random layout, hint and the board	Buttons are hidden and nothing happens	Pass

4.3 Other automated or manual tests not corresponding to the acceptance criteria (if any)

Number	Test Input	Expected Result	Class Name of the Test Code	Method Name of the Test Code
1	Click on the speaker button on the menu, setup, and game screens	Music mutes and unmutes without restarting, border turns red/green and icon changes	N/A, manu test	N/A for test, but corresponding classess/methods are (SoundManager.toggle_music, AudioButton.draw)
2	Click the theme button on any screen	Background cycles Green, Red, Wood adn the button letter changes (G,R, W)	N/A (manu test)	N/A, but corresponding classes/methods are PegSolitaireApp, theme_index, ThemeButton.draw
3	Click Hint? During a game, then with no legal moves left	Orange rings on every moveable peg and HINT: # pegs can be moved; rings clear after about 2 second	N/A manual test	N/A but corresponding methods/classes are PegSoltiaireApp.game_screen, HintButton
4	Premade-31 move solution on English 7 board (made with Claude)	Every move legal and the game ends with one peg and an Outstanding rating	TestWinningPlaythrough and manu	Test class(test_english_7_full_winning_game) and the manual test loads a saved json file with the same moves and replays it when loaded in the gui

5. **(Part 1) Describe how the class hierarchy in your design deals with the common and different requirements of the Manual Game and the Automated Game.**
   At least 1/2 page, excluding screenshots/diagrams (12pt, single-spaced), is required. (3 Points)

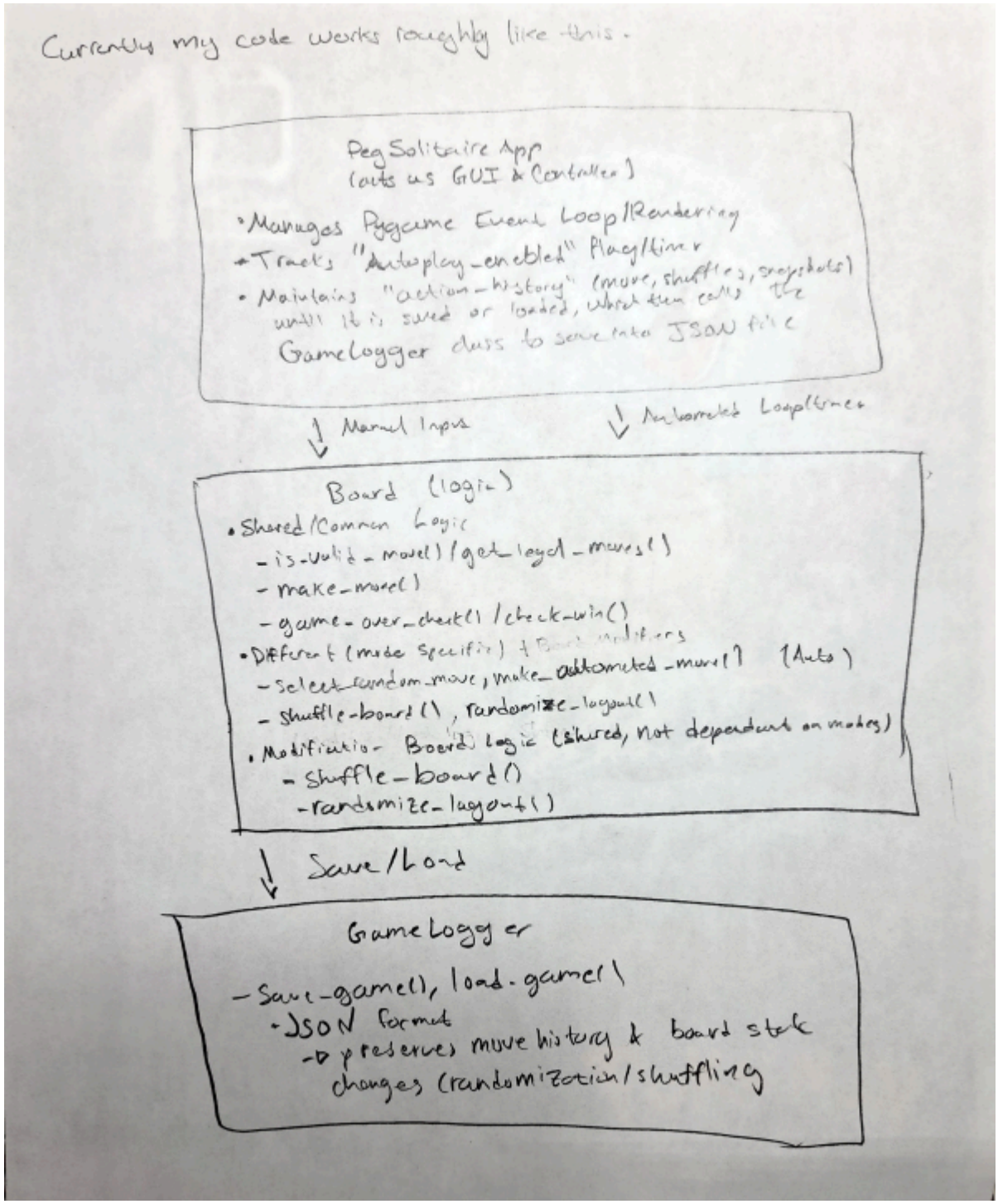
   In my design, instead of creating separate classes for manual and an automated mode, the core design logic is all kept within the same Board class, with the only real difference between them is where the moves come from. The reason why I designed it this way was that I did not want the manual vs automated mode to be a completely different game mode you select on game launch, but I wanted the difference to be a toggle. To implement this as easily as possible, I thought it was best to look at the fundamental difference between them, which is simply on how the move is generated. As mentioned earlier, what they have in common is the Board class, which is the central logic for this system. They share the same things across modes, such as board size, board type, checkers for valid/legal moves, making moves and deciding whether a game is over/won and their performance. Both of the mode types, whether computer generated or manual, go through the same logic and make_move process. Since the mode type is a toggle as well, they share the same game screen within the PegSolitaireApp class which draws the same board, runs the same game over/win check, same move recording process, and also messages/ratings. The Shuffle, random layout, hints and save/load features work the same as well.

   As mentioned before, the only real difference is where the move comes from. In manual mode, our moves are decided based on the player's clicks in which the game screen converts a selected peg and destination, which is validated through check/get_legal moves and then passed as make_move to then change the board state. Instead of player clicks, the moves for the automated mode come from the board. It has two main functions for this functionality, the first is select_random_move, which is the automated equivalent of a player click as it gets a list of all the legal moves from all the pegs and randomly selects one of the moves from there and returns it. Afterwards, the function Make_automated_move is almost identical to make_move which is used by the manual mode. It actually calls make_move within the function, but the only small difference is an extra safety check by checking whether there is none, but the core logic is fundamentally the same. The select_random move then passes it on to the gui/controller, which then makes it a "pending" move, which is so it can then highlight the move, run a small delay to show it on the screen, and then it calls make_automated_move from that pending move which is randomly selected. The only difference in terms of GUI interaction under the PegSolitaireApp class umbrella is the automated move button, which simply turns on or off the automated move and enables/disables the autoplay_enabled flag, which then determines how the move should be generated.

   All the other classes/objects are single purpose and small, not depending on the game mode. These include MenuButton, RadioButton, AudioButton, ThemeButton, HintButton, Save/Load Game Button, which work the same regardless of game mode. GameLogger also reports the moves the same way and makes no distinction (same with SoundManager class).

   The main tradeoff between these implementations for the mode difference is to have a simple flag within the controller rather than a class for each of the modes. It is good as we can simply keep the same logic that was already in the controller and simply add a flag to move where the move generation should go to, but the drawback could be the organization of it, as we could make a GameMode base class that keeps the same

overall logic, and then create a subclass of AutomatedMode and another for ManualMode, which would just vary by the get_next_move() function where the manual mode gets it from clicks while the automated mode gets it randomly. In hindsight, this approach seems to be better, but I had not planned as far in advance and the approach I took felt that it would take less code refactoring in order to implement it.



(Part 2) Demonstrate how you use LLM (ChatGPT or other) to analyze how well your code adheres to the design principles discussed in class - modularity, cohesion, coupling, and encapsulation – using the

definitions provided in class. Provide screenshots of your interactions with the LLM, showing the prompts used and the responses received.
Interpret the LLM's feedback, refine your prompts if needed to obtain more relevant responses, and discuss any changes made to your code based on the analysis. Explain how these refinements improve adherence to the design principles.
At least 1/2 page is required, excluding screenshots (12pt, single-spaced). (2 Points)

Based on the LLM results from Gemini, this is how the analysis went based on those 4 principles. The first is modularity. I gave it the definition that we used in class (slideshow), which is the degree to which the components of a system are separated. The analysis indicated that my code was doing well here, as we have high degree of modularity as I cleanly divided up the functions into their distinct files and classes. The reason why this is is that my core game logic is put within board.py, reusable interface elements (MenutButton, RadioButton, AudioButton, ThemeButton, HintButton) are encapsulated within components.py, persistence/gamelogger is handled within GameLogger class, audio management for the music in sound_manager.py, and the application flow, screen management and game loop is within the peg_solitaire_app.py. Overall, I do think this feedback is good, as I essentially have everything put within their appropriate and relevant task that puts the related functions all encapsulated within them (although this somewhat fits with cohesion).

The second one is cohesion. I again gave it the definition we used in class which is The degree to which the elements inside a module belong together. Gemini said that I had strong functional cohesion, as similar to with encapsulation, since I have them separated, everything within that separation (chunks) are related, as the Board class groups the methods that are just involved with evaluating and generating different board states, SoundManager strictly having grouped elements that handle the sound component and the muting/enabling of audio, and then the GameLogger being solely focused within it about saving and loading action history. Overall, I do agree with this assessment as everything is organized and the methods are relevant to the classes that are within them, giving me good cohesion in my program.

Afterwards we have coupling, which the definition is the degree of interdependency between modules. The LLM evaluated this as my weakest area, where there is a healthy balance between low to moderate interdependence. The reason for this being is that the spots that we do well where there is low coupling is like the Board class which is entirely interdependent of any UI or graphical rendering framework (it doesmn't even import pygame) which is what helps many of the core game rules being able to be cleanly tested within test_peg_solitaire.py. The area for refinement is the tighter dependencies within the main GUI controller, peg_solitiare_gui.py with the internal board state as it often has to update or restore grid snapshots for replays and shuffles. Overall, I definitely agre with this assessment. From some of my past experience with Java, we would typically have a controller folder which would help organize some of the slight flow and logic that needs to be done within the gui section, to just keep the gui section just about placing things on the screen. It isn't terrible as my other classes do well with cohesion, but this file in [articular is definitely the weakest. If I were to redo this or do a massive revamp, I would have planned it out better to have my own controller which would help reduce some of that interdependency with the gui and some of the decision making it does. Some of it is somewhat necessary however for some of my implementations, especially with the automated moves as I do incorporate some logic in there in order to make sure the pegs are highlighted when selected, but this isn't as big of a deal.

Lastly for encapsulation, there was some weakness here as well, which I agree with. This is due to some of the information being relaxed as peg_solitiare_gui.py often directly manipulate the raw internal structures of

the data, such as game_board.board directly instead of using mutators such as set and get within the board. Class.

Based on this I tried to work to reduce the interdependency between the modules and try to minimize it within the peg_solitaire_gui.py class, especially with the internal board start with update/restore for snapshots. And the other thing is to have set mutators within this file to improve the encapsulation.

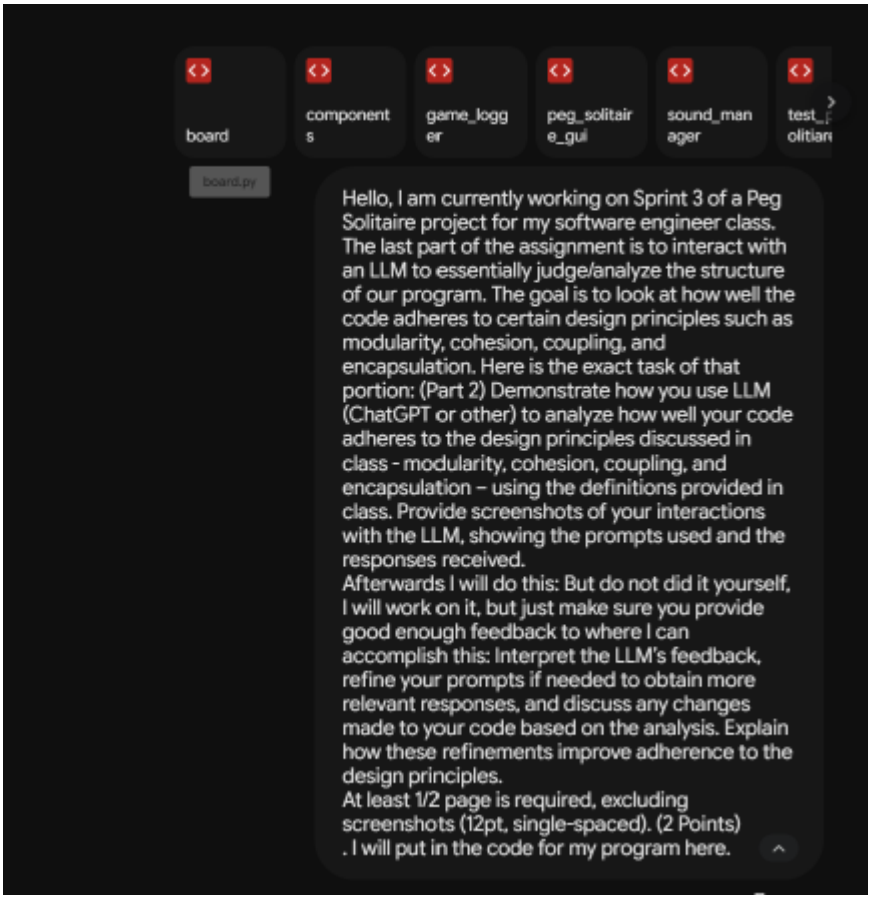
This is what I did to improve the encapsulation and coupling. I used these mutator functions in order to improve it. I did not add a set for the set grid value. The reason why I didn;t do this is that it is simply not necessary. The only real way to change the values should be by making a move or by randomization, which is already done well and kept within the board logic, so adding that extra function is unnecessary and may just provide possible risks. So I implemented this method and I changed the code in the gui/controller accordingly to use these to set the board grid or get the cell value instead of directly accessing the structure. Overall, this helps encapsulation as we are wrapping the internal structure with information hiding and access control, establishing a clear boundary. And for coupling what this does here by reducing the interdependency between the gui controller and the core game logic. Previously the GUI module tightly coupled the logic in a sense that the board was explicitly stored as a 2D list and directly mutated (game_board.board[row][col]). And for future refactoring, these changes as of now do a good job for encapsulation and reducing coupling, but the peg_solitaire_gui.py class does carry some dual responsible in terms of UI presentation and event flow. WHat I might introduce in the future is a GameController to handle more of the event logic and separate it from the UI presentation, more similar to a Java style controller. However given the time constraints with the sprint, I think I will consider tit for a future implementation where I can more carefully implement it.
Here is some of the code I implemented within the board.py class, of which I made appropriate changes within gui.py file where this applies.

```
# Mutator method - implemented as suggested by the LLM analysis
def set_board_grid(self, grid_snapshot):
    """Mutator method to update or restore the board grid state."""
    self.board = copy.deepcopy(grid_snapshot)

def get_cell(self, row, col):
    """Accessor method to retrieve the state of a specific cell."""
    return self.board[row][col]
```

Screenshots of LLM (Gemini)

Initial Prompt:



LLM response to first prompt:

Here is a comprehensive breakdown and structural guide based on your codebase (`board.py` , `components.py` , `game_logger.py` , `peg_solitaire_gui.py` , and `sound_manager.py`).

This guide is designed to give you all the analytical insights, example prompts, refinement strategies, and design principle evaluations you need to write your 1/2+ page report and document your LLM interaction screenshots.

1. Architectural Mapping to Design Principles

Before prompting an LLM, it helps to understand how your codebase currently aligns with the four key design principles:

- **Modularity:** Your project is exceptionally well-modularized by separating concerns into distinct files and classes:
  - `Board` (`board.py`): Pure game logic, rule validation, board layout generation, and state evaluation.
  - `GameLogger` (`game_logger.py`): Handles file I/O, JSON serialization, and persistence.
  - `SoundManager` (`sound_manager.py`): Manages audio tracks and volume toggles.
  - `components.py` : Reusable UI widgets (`MenuButton` , `RadioButton` , `AudioButton` , `ThemeButton` , `HintButton`).
  - `PegSolitaireApp` (`peg_solitaire_gui.py`): The central application controller managing screens and the main event loop.
- **Cohesion:** Classes exhibit **high cohesion.** For example, `Board` methods (`is_valid_move` , `make_move` , `shuffle_board` , `get_legal_moves`) all directly serve board state management. `SoundManager` exclusively handles audio operations.
- **Coupling:** There is **low-to-moderate coupling.** `Board` is completely decoupled from Pygame and UI frameworks (it doesn't import pygame), meaning game logic can be tested entirely headlessly via `unittest` . However, `PegSolitaireApp` tightly couples with Pygame events and directly inspects/modifies some board grid states during replays or shuffles.
- **Encapsulation:** Data and behavior are bundled nicely inside classes, though some attributes (like `game_board.board`) are accessed or modified externally across modules (e.g., during replay queue restoration or grid snapshots).

2. Recommended Initial LLM Prompt & Expected Feedback

To generate the screenshots required for Part 2 of your assignment, you can use a prompt like the one below in ChatGPT or another LLM, pasting your core classes or summarizing their structure:

Initial Prompt Example

> "Review the following Python classes for a Peg Solitaire game: `Board` (manages rules and grid state), `GameLogger` (handles JSON save/load), `SoundManager` (audio), reusable UI components in `components.py`, and `PegSolitaireApp` (main GUI controller). Analyze how well this architecture adheres to the design principles of ***modularity, cohesion, coupling, and encapsulation***, citing specific strengths and potential areas for improvement."

Expected Initial LLM Feedback Summary

- **Strengths:** Excellent separation of UI from business logic (keeping `Board` independent of Pygame); highly cohesive utility classes (`GameLogger`, `SoundManager`); clear component breakdown.
- **Potential Critique (Areas for Refinement):** `PegSolitaireApp` acts as a bit of a "god controller" handling too many responsibilities (rendering, event routing, state tracking, timers, replay queues); direct external access to `game_board.board` bypasses strict encapsulation.

3. Prompt Refinement & Iterative Analysis

For your write-up, your assignment asks you to explain how you **refined your prompts** to get more relevant responses and how those improvements impacted design principles.

Refined Prompt Example

> "Focus specifically on ***encapsulation and coupling*** between `PegSolitaireApp` and `Board`. Right now, the GUI directly inspects and overwrites `game_board.board` during replays, shuffles, and randomizations. Propose specific refactoring strategies (such as adding explicit accessor/mutator methods or state-reset methods in `Board`) to reduce coupling and strengthen encapsulation without breaking headless unit tests."

4. Discussing Code Changes & Adherence Improvements

When writing your report, you can discuss how you interpreted the LLM's feedback and what changes you made (or could make) to improve adherence:

1. **Improving Encapsulation:**
   - *Critique:* Direct manipulation of `game_board.board` from `peg_solitaire_gui.py` breaks information hiding.
   - *Refinement / Change:* Instead of setting `game_board.board = snapshot_grid` externally in the GUI controller, encapsulating this inside a `board.restore_grid(snapshot_grid)` method.
   - *Impact:* Protects internal representation and ensures the Board class controls its own state mutations.
2. **Reducing Controller Coupling:**
   - *Critique:* `PegSolitaireApp` handles a massive event loop and state machine.
   - *Refinement / Change:* Delegating sub-screens or state actions into dedicated helper handlers.
   - *Impact:* Improves maintainability and keeps coupling loose between the UI presentation layer and game rules.

Next Steps for Your Write-Up

To easily hit your 1/2 page requirement (12pt, single-spaced):

1. Take screenshots of your chat interaction with ChatGPT using the prompts provided above.
2. Structure your text into clear sections: **Initial LLM Interaction**, **LLM Feedback & Interpretation**, **Prompt Refinement**, and **Architectural Discussion / Code Improvements**.

Would you like help drafting the specific narrative paragraphs for your 1/2-page report based on this breakdown?

I then re-prompted:

No I do not need you to really help me take my screenshots or write my report. I will ask for clarification if needed. You are the LLM that I am using for this portion. Just focus on the initial prompt for the LLM and do your part please.

It then responded:

Here is a thorough architectural code review of your Peg Solitaire project, analyzing its structure against the design principles of **modularity, cohesion, coupling, and encapsulation.**

Architectural Code Review: Peg Solitaire Project

1. Modularity

- **Evaluation:** The project demonstrates **excellent modularity** by separating concerns across distinct files and classes. Core responsibilities are cleanly isolated:
  - `board.py` manages all underlying game rules, board generation, move validation, and state evaluation. board.py
  - `components.py` contains reusable, modular user interface widgets like `MenuButton`, `RadioButton`, `AudioButton`, `ThemeButton`, and `HintButton`. components.py
  - `game_logger.py` handles file input/output, JSON serialization, and persistence logic independently. game_logger.py
  - `sound_manager.py` encapsulates audio track loading and volume toggling. sound_manager.py
  - `peg_solitaire_gui.py` acts as the main application controller, managing screen transitions, event loops, and the active gameplay rendering pipeline. peg_solitaire_gui.py
- **Impact:** This clear separation makes the codebase clean, highly navigable, and easy to maintain or expand with new features.

2. Cohesion

- **Evaluation:** The classes exhibit **high functional cohesion**, meaning the methods grouped inside each class closely relate to a single, unified purpose:
  - The `Board` class groups methods that naturally belong to game state management, such as `is_valid_move`, `make_move`, `get_legal_moves`, `shuffle_board`, and `check_win`. board.py
  - `SoundManager` focuses entirely on audio mixer operations (`load`, `toggle_music`, volume setting). sound_manager.py
  - `GameLogger` statically handles saving game history and loading saved json files without mixing UI or game logic. game_logger.py
- **Impact:** Methods and attributes work together toward singular objectives, reducing side effects and making individual components predictable.

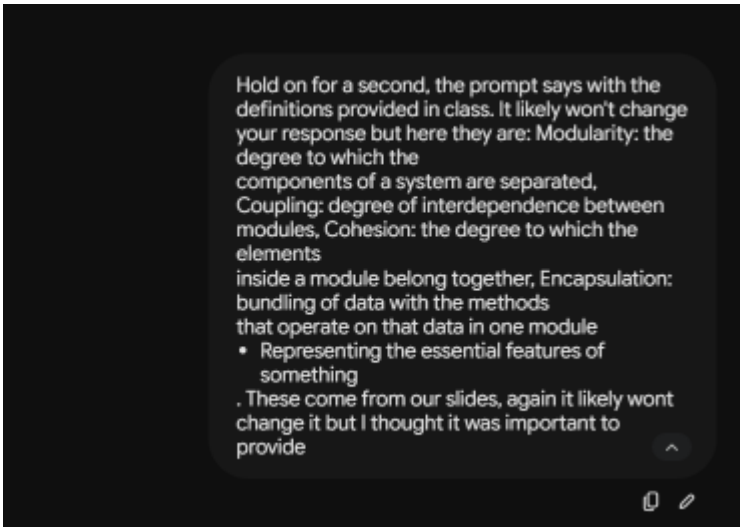
3. Coupling

- **Evaluation:** The codebase maintains **low-to-moderate coupling**:
  - *Low Coupling (Positive):* The `Board` class is completely decoupled from Pygame and UI rendering frameworks. Because `board.py` doesn't import graphical libraries, game logic can be tested entirely headlessly via `unittest` in `test_peg_solitiare.py`. board.py + 2
  - *Moderate Coupling (Areas for Refinement):* The main GUI controller (`PegSolitaireApp`) exhibits tighter coupling to internal board representations, occasionally accessing or overwriting grid states directly (e.g., during replay queues and board snapshot restorations). board.py + 1
- **Impact:** While headless testability is successfully preserved, reducing direct dependency on internal board data structures from the UI controller would make the system even more decoupled.

4. Encapsulation

- **Evaluation:** Data and behavior are successfully bundled inside objects across the application. However, encapsulation could be slightly tightened:
  - The `Board` class bundles grid layouts and state rules well, but external modules directly read and modify `game_board.board` during actions like replays, shuffles, and randomizations. board.py + 1
  - Exposing the raw 2D list `board` externally bypasses strict information hiding.
- **Impact:** Encapsulation is strong for standard gameplay flows, but providing explicit methods inside `Board` (such as `restore_grid(snapshot)`) rather than direct list assignment would better protect internal representation.

I then re-prompted one more time, to make it more according with the definitions we used in class, which is what I ended up mainly using for the analysis:



Thank you for providing those specific class definitions. Incorporating your exact course definitions, here is the structured analysis evaluating your codebase against **modularity, cohesion, coupling, and encapsulation**:

Architectural Code Review: Peg Solitaire Project

1. Modularity

- *Definition:* The degree to which the components of a system are separated.
- *Analysis:* Your system achieves a high degree of modularity by cleanly separating distinct functional responsibilities into individual files and classes:
  - Core game rules and grid management are isolated in `board.py`. board.py
  - Reusable interface elements (`MenuButton`, `RadioButton`, `AudioButton`, `ThemeButton`, `HintButton`) are encapsulated in `components.py`. components.py
  - Persistence and JSON file I/O operations are handled independently in `game_logger.py`. game_logger.py
  - Audio management is separated into `sound_manager.py`. sound_manager.py
  - Application flow, screen management, and the main event loop reside in `peg_solitaire_gui.py`. peg_solitaire_gui.py

2. Cohesion

- *Definition:* The degree to which the elements inside a module belong together.
- *Analysis:* Your classes exhibit strong functional cohesion, meaning the methods and attributes within each module closely support a single, unified responsibility:
  - The `Board` class groups methods that strictly belong to managing board states, validating jumps (`is_valid_move`, `make_move`), and tracking win/loss conditions (`check_win`, `game_over_check`). board.py
  - `SoundManager` groups elements dedicated entirely to loading audio files and toggling volume states. sound_manager.py
  - `GameLogger` contains static methods focused exclusively on saving and loading game action histories. game_logger.py

3. Coupling

- *Definition:* The degree of interdependence between modules.
- *Analysis:* The codebase maintains a healthy balance of low-to-moderate interdependence:
  - *Low Coupling (Strength):* The `Board` class is entirely independent of any UI or graphical rendering framework (it does not import Pygame). This low interdependence allows core game rules to be tested completely headlessly using `unittest` in `test_peg_solitiare.py`. board.py + 1
  - *Moderate Interdependence (Area for Refinement):* A tighter dependency exists between the main GUI controller (`peg_solitaire_gui.py`) and internal board state, where the GUI occasionally reaches directly into `game_board.board` to update or restore grid snapshots during replays and shuffles. board.py + 1

4. Encapsulation

- *Definition:* Bundling of data with the methods that operate on that data in one module.
- *Analysis:* Data fields (such as board dimensions, tracking states, and configurations) are successfully bundled with their behavioral methods inside classes like `Board`, `GameLogger`, and `SoundManager`. However, information hiding is slightly relaxed in practice because external controllers (like `peg_solitaire_gui.py`) directly manipulate raw internal structures (such as modifying `game_board.board` directly) rather than channeling all state changes strictly through dedicated mutator methods on the `Board` class. board.py + 4